- CAT1 :
- module1: full
- module 2 : full, except stream cipher, RC4 & random bit generation
- module 3 : RSA , Elgamal Cryptosystems

Topics
Modular arithmetic, Euclidean Algorithm, Finite Fields and Primitive Roots, Discrete Logarithms
AES, IDEA
Symmetric key cryptographic techniques, Classical ciphers (Ceaser/Vigenère, Hill, Playfair/Row transposition/Rail fence)
Fermat's and Euler's theorem, Chinese Reminder theorem
DES, Block Cipher modes of operation
Asymmetric key cryptographic techniques: principles, RSA, Elgamal Cryptosystems

DATA ENCRYPTION STANDARD (DES)

Introduction to DES

- most widely used block cipher in the world
- Symmetric key
- adopted in 1977 by NIST
- encrypts 64-bit data using 56-bit key
- has widespread use
- has been considerable controversy over its security
- After 2013, ***deprecated*** due to short key

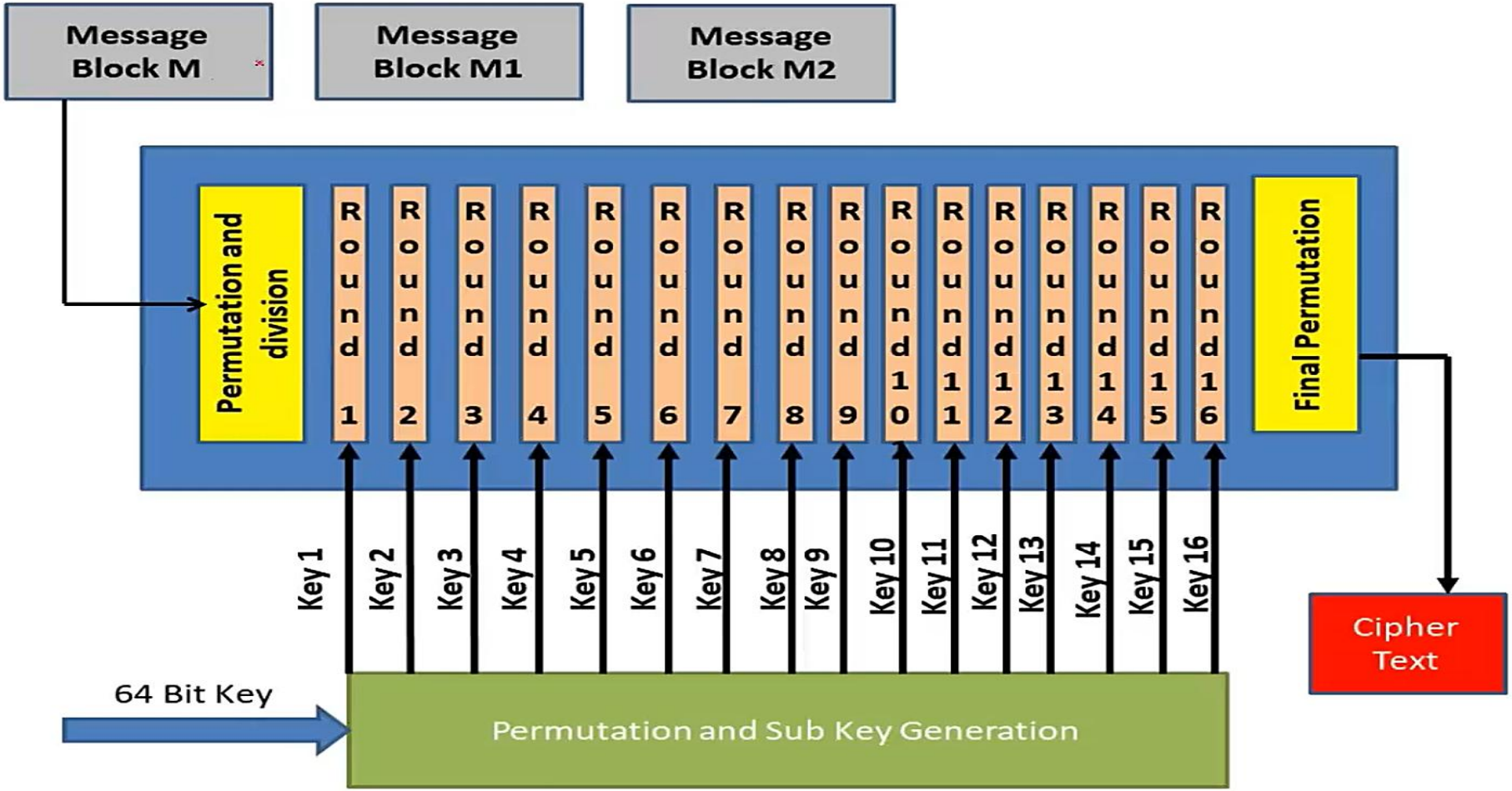
DES Block Cipher

For DES, data are encrypted in 64-bit blocks using a 56-bit key. The algorithm transforms 64-bit input(Plaintext) in a series of steps into a 64-bit output (Ciphertext). The same steps, with the same key, are used to reverse the encryption (Symmetric key cryptosystem).

The general structure of the DES consists of

1. **key schedule**
2. **initial permutation**
3. **round function**
4. **final permutation**

DES Encryption Overview



(1) Key schedule (generator): generating 16 subkeys

This algorithm generates the subkeys ($K \rightarrow K_1, K_2 \ldots K_{16}$).

1. **The 56 bits of the key are selected from the initial 64 bits by the Permuted Choice-1 (PC1) table.**

2. **The 56 bits are divided into two 28-bit halves.**

3. **In each round, both halves are rotated left by one or two bits (specified for each round).**

4. **The 48 subkey bits are selected by Permuted Choice 2 (PC2) table (24 bits from the left half, and 24 from the right) and used in each round.**

DES Block Cipher

(2) initial permutation:

Step 1: Plaintext is broken into blocks of length 64 bits.

Step 2: The 64-bit block undergoes an initial permutation (IP) using the initial permutation IP table, IP(M).

Step 3: The 64-bit permuted input is divided into two 32-bit blocks: left (L) and right (R). The initial values of the left and right blocks are denoted L_0 and R_0.

(3) round function:

Step 4: The function F(.) represents the heart of the DES algorithm. This function implements the following operations:

1-Expansion: The right 32-bit half-block is expanded to 48 bits using the expansion permutation (E) table, $E(R_{n-1})$.

2-Key mixing: The expanded result is combined with a subkey using an XOR operation. Sixteen 48-bit subkeys (one for each round) are derived from the main key using the key schedule, $K_n + E(R_{n-1})$. '+' denotes XOR operation

3-Substitution: After mixing in the subkeys, the block is divided into eight 6-bit pieces and fed into the substitution boxes (S-boxes), which implements nonlinear transformation. Each 6-bit piece uses as an address in the S-boxes where the first and last bits are used to address the ith row and the middle four bits to address the jth column in the S-boxes. The output of each S-box is 4-bit length piece. The output of all eight S-boxes is then combined into 32 bit section.

$K_n + E(R_{n-1}) = B_1B_2B_3B_4B_5B_6B_7B_8$

$S(Kn + E(Rn-1)) = S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$

4-Permutation: The 32 bits outputs from the S-boxes are rearranged using the P-box, $F=P(S(K_n + E(R_{n-1})))$

5: There are 16 rounds of operations on the L and R blocks. During each round, the following formula is applied:

$$L_n = R_{n-1}$$

$$R_n = L_{n-1} \text{ XOR } F(R_{n-1}, K_n) \quad \text{where } n=1, 2, 3, \ldots, 16$$

(3) Final Permutation:

Step 6: The results from the final DES round (i.e., L_{16} and R_{16}) are recombined into a 64-bit value and rearranged using an inverse initial permutation (IP^{-1}) table. The output from IP^{-1} is the 64-bit ciphertext block.

Example

K = 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001

M = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111

(1) Key schedule (generator): generating 16 subkeys

This algorithm generates the subkeys ($K \rightarrow K_1, K_2 \ldots K_{16}$).

1. **The 56 bits of the key are selected from the initial 64 bits by the Permuted Choice-1 (PC1) table.**

2. **The 56 bits are divided into two 28-bit halves.**

3. **In each round, both halves are rotated left by one or two bits (specified for each round).**

4. **The 48 subkey bits are selected by Permuted Choice 2 (PC2) table (24 bits from the left half, and 24 from the right) and used in each round.**

K = 00010011 00110100 01010111 01111001 10011011 10111100 11011111 11110001

1. The 56 bits of the key are selected from the initial 64 by Permuted Choice-1 (PC1) table.

We get the 56-bit new permutation key

K+ = 1111000 0110011 0010101 0101111 0101010 1011001 1001111 0001111

2. The 56 bits are divided into two 28-bit halves.

C0 = 1111000 0110011 0010101 0101111

D0 = 0101010 1011001 1001111 0001111

3. In each round, both halves are rotated left by one or two bits using the Schedule of Left Shits Table

C_1 = 1110000110011001010101011111

D_1 = 1010101011001100111100011110

C_2 = 1100001100110010101010111111

D_2 = 0101010110011001111000111101

C_3 = 0000110011001010101011111111

D_3 = 0101011001100111100011110101

C_4 = 0011001100101010101111111100

D_4 = 0101100110011110001111010101

C_5 = 1100110010101010111111110000

D_5 = 0110011001111000111101010101

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

(d) Schedule of Left Shifts

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

C_6 = 0011001010101011111111000011
D_6 = 1001100111100011110101010101

C_7 = 1100101010101111111100001100
D_7 = 0110011110001111010101010110

C_8 = 0010101010111111110000110011
D_8 = 1001111000111101010101011001

C_9 = 0101010101111111100001100110
D_9 = 0011110001111010101010110011

C_{10} = 0101010111111110000110011001
D_{10} = 1111000111101010101011001100

C_{11} = 0101011111111000011001100101
D_{11} = 1100011110101010101100110011

C_{12} = 0101111111100001100110010101
D_{12} = 0001111010101010110011001111

C_{13} = 0111111110000110011001010101
D_{13} = 0111101010101011001100111100

C_{14} = 1111111000011001100101010101
D_{14} = 1110101010101100110011110001

C_{15} = 1111100001100110010101010111
D_{15} = 1010101010110011001111000111

C_{16} = 1111000011001100101010101111
D_{16} = 0101010101100110011110001111

Generating 16 subkeys . . .

4. The 48 subkey bits are selected by Permuted Choice 2 (PC2) table (24 bits from the left half, and 24 from the right) and used in each round.

C_1 = 1110000110011001010101011111
D_1 = 1010101011001100111100011110

C_1D_1 = 1110000 1100110 0101010 1011111 1010101 0110011 0011110 0011110

The first Subkey(48 bits)

K_1 = 000110 110000 001011 101111 111111 000111 000001 110010

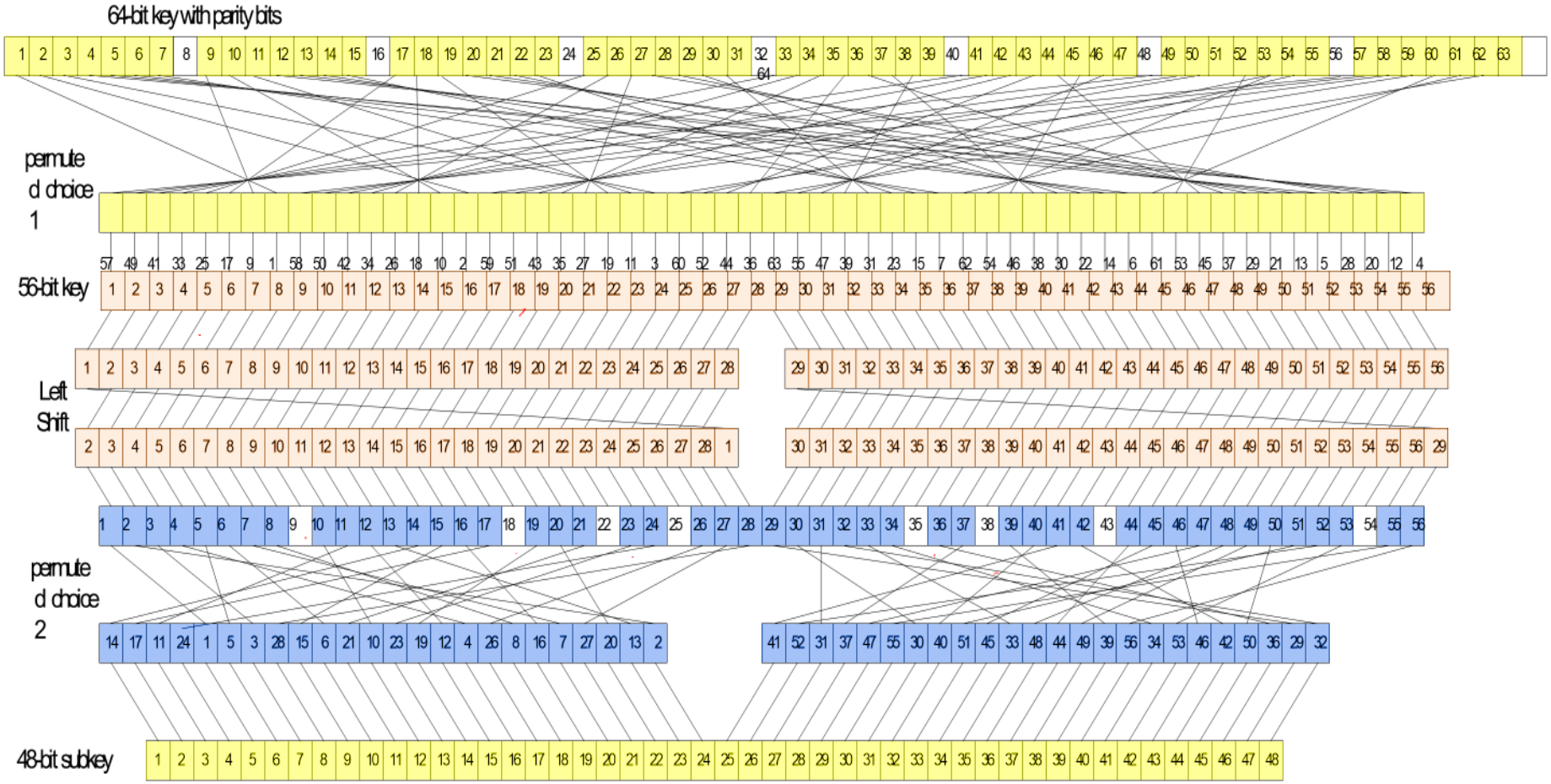
Similarly the other Subkey(48 bits)

K_2 = 011110 011010 111011 011001 110110 111100 100111 100101
K_3 = 010101 011111 110010 001010 010000 101100 111110 011001
K_4 = 011100 101010 110111 010110 110110 110011 010100 011101
K_5 = 011111 001110 110000 000111 111010 110101 001110 101000
K_6 = 011000 111010 010100 111110 010100 000111 101100 101111
K_7 = 111011 001000 010010 110111 111101 100001 100010 111100
K_8 = 111101 111000 101000 111010 110000 010011 101111 111011
K_9 = 111000 001101 101111 101011 111011 011110 011110 000001
K_{10} = 101100 011111 001101 000111 101110 100100 011001 001111
K_{11} = 001000 010101 111111 010011 110111 101101 001110 000110
K_{12} = 011101 010111 000111 110101 100101 000110 011111 101001
K_{13} = 100101 111100 010111 010001 111110 101011 101001 000001
K_{14} = 010111 110100 001110 110111 111100 101110 011100 111010
K_{15} = 101111 111001 000110 001101 001111 010011 111100 001010
K_{16} = 110010 110011 110110 001011 000011 100001 011111 110101

PC2 table

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

DES Key Schedule … Contd.



DES Block Cipher

(2) initial permutation:

Step 1: Plaintext is broken into blocks of length 64 bits.

Step 2: The 64-bit block undergoes an initial permutation (IP) using the initial permutation IP table, IP(M).

Step 3: The 64-bit permuted input is divided into two 32-bit blocks: left (L) and right (R). The initial values of the left and right blocks are denoted L_0 and R_0.

1. Plaintext is broken into blocks of length 64 bits

M = 0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

2. The 64-bit block undergoes an initial permutation (IP) using the initial permutation IP table, IP(M).

IP = 1100 1100 0000 0000 1100 1100 1111 1111 1111 0000 1010 1010 1111 0000 1010 1010

3. The 64-bit permuted input is divided into two 32-bit blocks: left (L) and right (R). The initial values of the left and right blocks are denoted L_0 and R_0

L_0 = 1100 1100 0000 0000 1100 1100 1111 1111

R_0 = 1111 0000 1010 1010 1111 0000 1010 1010

(3) round function:

Step 5: The function F(.) represents the heart of the DES algorithm. This function implements the following operations:

1-Expansion: The right 32-bit half-block is expanded to 48 bits using the expansion permutation (E) table, $E(R_{n-1})$.

2-Key mixing: The expanded result is combined with a subkey using an XOR operation. Sixteen 48-bit subkeys (one for each round) are derived from the main key using the key schedule, $K_n + E(R_{n-1})$. '+' denotes XOR operation

3-Substitution: After mixing in the subkeys, the block is divided into eight 6-bit pieces and fed into the substitution boxes (S-boxes), which implements nonlinear transformation. Each 6-bit piece uses as an address in the S-boxes where the first and last bits are used to address the ith row and the middle four bits to address the jth column in the S-boxes. The output of each S-box is 4-bit length piece. The output of all eight S-boxes is then combined into 32 bit section.

$K_n + E(R_{n-1}) = B_1B_2B_3B_4B_5B_6B_7B_8$

$S(Kn + E(Rn-1)) = S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$

4-Permutation: The 32 bits outputs from the S-boxes are rearranged using the P-box, $F=P(S(K_n + E(R_{n-1})))$

5: There are 16 rounds of operations on the L and R blocks. During each round, the following formula is applied:

$$L_n = R_{n-1}$$

$$R_n = L_{n-1} \text{ XOR } F(R_{n-1}, K_n) \quad \text{where } n=1, 2, 3, \ldots, 16$$

R_0 = 1111 0000 1010 1010 1111 0000 1010 1010

E BIT-SELECTION TABLE

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

$\mathbf{E}(R_0)$ = 011110 100001 010101 010101 011110 100001 010101 010101

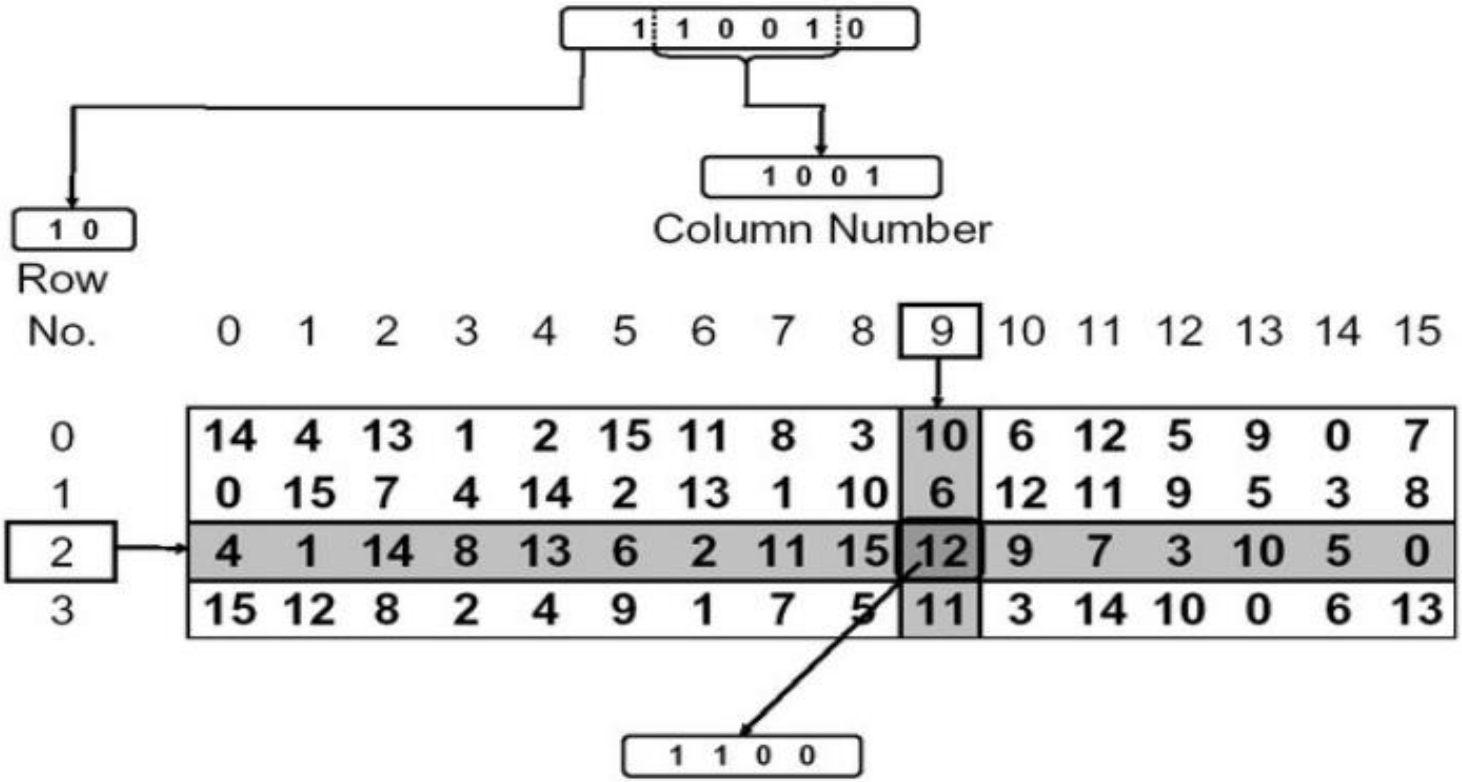
$K_1+\mathbf{E}(R_0)$ = 011000 010001 011110 111010 100001 100110 010100 100111.

K1+ E(R0) = ***B1*** ***B2*** ***B3*** ***B4*** ***B5*** ***B6*** ***B7*** ***B8***

We now calculate

$S_1(B_1)\ S_2(B_2)\ S_3(B_3)\ S_4(B_4)\ S_5(B_5)\ S_6(B_6)\ S_7(B_7)\ S_8(B_8)$

= 0101 1100 1000 0010 1011 0101 1001 0111



Application of S-box in DES Algorithm

The DES S-Boxes

S_1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

$f(R_0,K_1)$ - Final stage of *f- Permutation*

$S_1(B_1)S_2(B_2)S_3(B_3)S_4(B_4)S_5(B_5)S_6(B_6)S_7(B_7)S_8(B_8)$

= 0101 1100 1000 0010 1011 0101 1001 0111

$f(R_0,K_1)$ **= 0010 0011 0100 1010 1010 1001 1011 1011**

P

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

L_0 = 1100 1100 0000 0000 1100 1100 1111 1111

R_0 = 1111 0000 1010 1010 1111 0000 1010 1010

$f(R_0,K_1)$ **= 0010 0011 0100 1010 1010 1001 1011 1011**

$L_n = R_{n-1}$

$R_n = L_{n-1} + f(R_{n-1},K_n)$

Let + denote XOR addition

For n = 1, we have

K_1 = 000110 110000 001011 101111 111111 000111 000001 110010

$L_1 = R_0$ = 1111 0000 1010 1010 1111 0000 1010 1010

$R_1 = L_0 + f(R_0,K_1)$

R1 = 1110 1111 0100 1010 0110 0101 0100 0100

(3) Final Permutation:

Step 6: The results from the final DES round (i.e., L_{16} and R_{16}) are recombined into a 64-bit value and rearranged using an inverse initial permutation (IP^{-1}) table. The output from IP^{-1} is the 64-bit ciphertext block.

Finally after 16 Rounds

Output of 16 Rounds

L_{16} = 0100 0011 0100 0010 0011 0010 0011 0100
R_{16} = 0000 1010 0100 1100 1101 1001 1001 0101

We reverse the order of these two blocks and apply the final permutation to
$R_{16}L_{16}$ = 00001010 01001100 11011001 10010101 01000011 01000010 00110010 00110100

IP^{-1} = 10000101 11101000 00010011 01010100 00001111 00001010 10110100 00000101

IP^{-1}

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(c) Expansion Permutation (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

(b) Inverse Initial Permutation (IP^{-1})

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

(d) Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

Tables used in the DES algorithm

The DES S-Boxes

S_1	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13
S_2	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9
S_3	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12
S_4	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14
S_5	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3
S_6	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13
S_7	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12
S_8	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

(a) Input Key

1	2	3	4	5	6	7	8
9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24
25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56
57	58	59	60	61	62	63	64

(b) Permuted Choice One (PC-1)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

(c) Permuted Choice Two (PC-2)

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

(d) Schedule of Left Shifts

Round Number	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Bits Rotated	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Tables used in DES key generator

General remarks in the DES:

1- The S-boxes provide the core of the security of DES and the cipher would be linear, and trivially breakable without them.

2- The substitution and permutation in the DES provide confusion and diffusion.

DES Block Cipher

DES decryption :

The decryption algorithm uses the same steps exactly as in the encryption algorithm except that the application of the subkeys is reversed (i.e. in round1 use K_{16}, round2 use K_{15} and so on).

Security and cryptanalysis:

The two most widely used attacks on block ciphers are linear and differential cryptanalysis. DES is also vulnerable to a brute-force (exhaustive search) attack.

Triple DES:

In cryptography, Triple DES (3DES or TDES), officially the Triple Data Encryption Algorithm (TDEA or Triple DEA), is a symmetric-key block cipher, which applies the DES cipher algorithm three times to each data block.

Therefore, Triple DES uses a "key bundle" that comprises three DES keys, $K1$, $K2$ and $K3$, each of 56 bits.
The encryption algorithm is:

$$\text{ciphertext} = E_{K3}(D_{K2}(E_{K1}(\text{plaintext}))).$$

That is, DES encrypt with $K1$, DES *decrypt* with $K2$, then DES encrypt with $K3$.
Decryption is the reverse:

$$\text{plaintext} = D_{K1}(E_{K2}(D_{K3}(\text{ciphertext}))).$$

That is, decrypt with $K3$, *encrypt* with $K2$, then decrypt with $K1$.
Each triple encryption encrypts one block of 64 bits of data.

Ex- 1 S-DES Block Cipher

Assuming the output of the first round of f_k os S-DES is 00101011 and K_2 is 10111001, what is the output ciphertext? You may use the information below:

IP: 2 6 3 1 4 8 5 7 E/P: 4 1 2 3 2 3 4 1 P4: 2 4 3 1

$$S_0 = \begin{bmatrix} 01 & 00 & 11 & 10 \\ 11 & 10 & 01 & 00 \\ 00 & 10 & 01 & 11 \\ 11 & 01 & 11 & 10 \end{bmatrix} \quad S_1 = \begin{bmatrix} 00 & 01 & 10 & 11 \\ 10 & 00 & 01 & 11 \\ 11 & 00 & 01 & 00 \\ 10 & 01 & 00 & 11 \end{bmatrix}$$

Note: you need to determine IP^{-1} yourself

Solution:

Output(first round)=00101011 K2: 10111001

Swap halves: 10110010

E/P on right half: 00010100

XOR with K2: 10101101

Output of S0: 10

Output of S1: 00

P4: 0001 XOR with left half: 1010

IP^{-1} [4 1 3 5 7 2 8 6]: 01101000

Ciphertext: 01101000